834 EDI File Compare — User Guide

A practical, step-by-step guide to installing, running, and using the 834 EDI File Compare tool — from your very first run to reading results and exporting a report.

> What this tool does: compares two 834 EDI benefit enrollment files (a baseline and a comparison), groups the records by subscriber, and shows you exactly what changed — who was added, removed, or modified — plus any header/envelope differences. It is local-only: no data ever leaves your machine.

New here? Start at 1. Prerequisites and work down. In a hurry? Jump to 5. Quick start with the sample files.

Table of contents

1. Prerequisites
2. Installing the project
3. Running the application
4. Using the application (GUI) - 4.1 The main window - 4.2 Choosing your two files - 4.3 Running a comparison - 4.4 Reading the results tree - 4.5 Reading the detail pane - 4.6 "Show all segments" - 4.7 Exporting to Excel
5. Quick start with the sample files
6. The Excel report, sheet by sheet
7. Command-line (headless) usage
8. How the comparison actually works
9. Troubleshooting
10. Tips for large files
11. Glossary
12. Regenerating this guide (PDF)

1. Prerequisites

Requirement	Notes
Python 3.12 or newer	3.12, 3.13, 3.14, 3.15 all work. Check with `python --version`.
tkinter	Part of the standard library. On most Windows/macOS installs it is already present. On some Linux distros you may need to install it (see below).
openpyxl	Used for Excel export. Installed via `requirements.txt`.
pytest	Only needed if you want to run the test suite.

Check your Python version:

```
python --version
```

> Linux note: if you get `No module named 'tkinter'`, install it for your distribution, e.g. `sudo apt install python3-tk` (Debian/Ubuntu) or `sudo dnf install python3-tkinter` (Fedora).

2. Installing the project

Work from the project folder (the one containing `app.py`).

Option A — use a virtual environment (recommended):

```
python -m venv .venv
```

Activate it, then install dependencies:

- Windows (PowerShell): `.venv\Scripts\Activate.ps1 pip install -r requirements.txt`

- Windows (Command Prompt): `.venv\Scripts\activate.bat pip install -r requirements.txt`
- macOS / Linux: `source .venv/bin/activate pip install -r requirements.txt`

Option B — install into your current environment:

```
pip install -r requirements.txt
```

That's it. There is nothing else to build or configure.

> Tip: if you already have a `.venv` in the project, just activate it and skip the `python -m venv` step.

---

3. Running the application

With your environment activated, start the app from the project folder:

```
python app.py
```

A window titled "834 EDI File Compare" opens. That is the whole application — no installation, no services, no network.

> Prefer the command line? See Section 7 for running a comparison without the GUI (useful for scripting or batch runs).

---

4. Using the application (GUI)

4.1 The main window

The window is divided into three areas, top to bottom:

```
+-------------------------------------------------------------------------+
|  File A (baseline):   [____________________]  [Browse…]  <- top bar    |
|  File B (comparison): [____________________]  [Browse…]                 |
|                                                                         |
|  [ ] Show all segments   [Compare]  [Export to Excel…]   <- status      |
|                                                                         |
|  +-------------------------------------------------------------+       |
|  |  Status | Subscriber / Field | Key | Detail                 |       |
|  |  (results tree — one row per subscriber, expandable)         |       |
|  +-------------------------------------------------------------+       |
|                                                                         |
|  +- Detail ---------------------------------------------------+         |
|  |  (side-by-side segments for the selected subscriber)       |         |
|  +-------------------------------------------------------------+       |
+-------------------------------------------------------------------------+
```

- Top bar — where you point at the two files.
- Options row — the "Show all segments" checkbox, the Compare and Export to Excel… buttons, and a status message on the right.
- Results tree — the list of subscribers (and header differences).
- Detail pane — the side-by-side segment view for whatever you select.

4.2 Choosing your two files

1. Click Browse… next to File A (baseline).
2. In the file dialog, pick your baseline 834 file (the "before" / reference).
3. Click Browse… next to File B (comparison).
4. Pick your comparison 834 file (the "after" / new extract).

Notes: - You can also type or paste a full path directly into either box. - Baseline vs. comparison convention: File A is treated as the reference. A subscriber that exists in A but not in B is Removed; one that exists in B but not in A is Added. If you pick them the other way round, Added and Removed swap — the underlying differences are the same, just viewed from the other side. - The file dialog suggests `*.834`, `*.txt`, and `*.edi`, but you can choose any file (the parser detects the delimiters from the file itself).

4.3 Running a comparison

Press Compare.

- If the files are valid, the results tree populates and the status message on the right shows a one-line summary, e.g.: `5 subscribers · 2 changed · 1 added · 1 removed · 6 header diffs`.
- Changed, added, and removed subscribers are auto-expanded so the differences are visible immediately.
- If a file can't be read or parsed, you get a clear error dialog (see Troubleshooting).

4.4 Reading the results tree

The tree has four columns: Status, Subscriber / Field, Key, and Detail.

Top level — one row per subscriber, colour-coded by status:

Colour	Status	Meaning
Amber	Changed	Present in both files, but something inside differs.
Green	Added	Present in File B only (new in the comparison).
Red	Removed	Present in File A only (gone in the comparison).
Grey	Unchanged	Present in both files, identical.

- Status — the status from the table above.
- Subscriber / Field — a friendly label: the person's name plus their identifying key, e.g. `Smith John [EMP00123]`.
- Key — the subscriber key used for matching: the `REF*17` value if present, otherwise the `NM1*IL` identity (shown as `Last|First|MemberID`).
- Detail — a compact summary, e.g. `A=12 segs, B=13 segs · 1 added, 1 changed`.

Expanding a subscriber (click the > arrow) reveals its segment-level differences as child rows. Each child row shows: - Status — `changed`, `added`, or `removed` (for that segment). - Subscriber / Field — the context, i.e. which block the segment belongs to: `Subscriber`, `Dependent <name>`, or `Coverage`. - Detail — what changed, e.g. `el4 'PLAN1' -> 'PLAN2'`.

Header Differences group. If the two files differ in their envelope/header (ISA/GS/ST control numbers, dates, BGN reference, etc.), a special Header Differences group appears at the top of the tree. Expand it to see each differing field with its value in File A and File B, e.g. `'000000001' -> '000000002'`. This is the place to look for "envelope-only" changes that are usually expected between two runs of the same extract.

4.5 Reading the detail pane

Click any subscriber row to load its detail into the Detail pane at the bottom. The pane is plain text, laid out in two sections:

1. Differences — a list of every difference for that subscriber: ``` Smith John [EMP00123] [Changed] key=EMP00123 (REF*17)

Differences: [changed] Coverage A: HD021HLTLAN1EMP B: HD021HLTLAN2EMP el4 'PLAN1' -> 'PLAN2' [added] Coverage B: DTP349D8*20261231 ``` Each entry shows the segment on side A and/or side B, plus a short note about exactly which element(s) changed.

2. All segments (A | B) — a full side-by-side of every segment for that subscriber, grouped by context (`Subscriber`, `Dependent …`, `Coverage`). This section appears when the subscriber is changed or when you have enabled "Show all segments" (see below).

> Selecting a child (segment) row also shows its parent subscriber's detail, so you can drill into a specific difference and still see the whole picture.

4.6 "Show all segments"

By default the tool keeps the view focused on what changed. The Show all segments (not just changed) checkbox changes that:

- Unchecked (default): the detail pane shows differences, and the full side-by-side only for subscribers that actually changed. This keeps large files fast to scan.
- Checked: the detail pane shows the complete side-by-side for any subscriber you select, including unchanged ones. Use this when you want to eyeball a specific subscriber's full record.

Toggling the checkbox immediately re-renders the detail for the currently selected subscriber — no need to re-run Compare.

4.7 Exporting to Excel

1. Run a comparison first (you must press Compare before exporting).
2. Press Export to Excel… and choose a save location and filename. - The suggested name is `834_compare_<A>_vs_<B>.xlsx`.
3. A confirmation dialog shows the saved path.

The workbook contains four sheets — see Section 6. The All Segments sheet reflects the "Show all segments" setting at the moment you export.

---

5. Quick start with the sample files

The `samples/` folder ships with synthetic 834 files (no real data). `sample_a.834` and `sample_b.834` are a matched pair with deliberate differences — the fastest way to see the tool work end to end.

1. Start the app: `python app.py`
2. File A: `samples/sample_a.834`
3. File B: `samples/sample_b.834`
4. Press Compare.

You should see a status line like: `5 subscribers · 2 changed · 1 added · 1 removed · 6 header diffs`

Expected results:

Subscriber	Status	Why
Smith John `[EMP00123]`	Changed	plan changed `PLAN1 -> PLAN2` (+ a termination date added)
Lopez Maria `[EMP00124]`	Changed	a new dependent (Mateo) was added
Chen Wei `[EMP00125]`	Removed	present in A, not in B
Okafor Ada	Unchanged	(no `REF*17`; matched by `NM1*IL`)
Vargas Luis `[EMP00127]`	Unchanged	(dependent with no member ID)
Patel Aisha `[EMP00126]`	Added	present in B, not in A

And a Header Differences group showing the envelope-only changes (GS/ST control numbers and dates).

> Note the subtle case: in File B, `Smith John`'s `REF*17` segment is in a different position than in File A. The tool still reports only the genuine plan change — the reorder is correctly ignored.

To generate fresh sample fixtures at any time (still no real data):

```
python make_fixtures.py
```

---

6. The Excel report, sheet by sheet

Sheet	What it contains	When to use it
Summary	File names, counts (subscribers, changed/added/removed/unchanged), segment-level diff count, header diff count, an overall IDENTICAL / DIFFERENT verdict, and any warnings.	First stop — the at-a-glance result.
Header Differences	One row per differing header field: `Field`, `File A`, `File B`.	Confirming envelope/control-number changes.
Subscriber Differences	One row per segment-level difference, grouped by subscriber: `Subscriber`, `Status`, `Key`, `Context`, `Type`, `Segment A`, `Segment B`, `Detail`. Added/removed whole subscribers get a single `(whole subscriber)` row.	The working list of what to review.
All Segments	A full side-by-side of every segment for every subscriber: `Subscriber`, `Status`, `Context`, `Segment A`, `Segment B`.	Deep dives / audit trail.

The report is deterministic: the same two files always produce the same workbook (no embedded timestamps in the data, no random styling).

---

7. Command-line (headless) usage

You can run a comparison without opening the GUI — handy for scripting, batch processing, or CI.

Basic comparison (prints a summary to the console):

```
python app.py "path\to\baseline.834" "path\to\comparison.834"
```

Example output:

```
File A : sample_a.834  (5 subscribers)
File B : sample_b.834  (5 subscribers)
Header diffs : 6
Changed      : 2
Added        : 1
Removed      : 1
Unchanged    : 2
Segment diffs: 3
  [Changed  ] Smith John [EMP00123]  (2 diffs)
  [Changed  ] Lopez Maria [EMP00124]  (1 diffs)
  [Removed  ] Chen Wei [EMP00125]  (0 diffs)
  [Added    ] Patel Aisha [EMP00126]  (0 diffs)
```

Also write the Excel report:

```
python app.py baseline.834 comparison.834 --out report.xlsx
```

Options:

Option	Effect
`file_a`	(positional) the baseline 834 file
`file_b`	(positional) the comparison 834 file
`--out PATH`	also write the Excel report to `PATH`

The process exit code is `0` on success and `1` if a file can't be read or parsed, so it plays nicely with scripts.

> Rule of thumb: running `python app.py` with no arguments opens the GUI; with any arguments it runs headless.

---

8. How the comparison actually works

A short, plain-English explanation of the matching logic, so you can trust the results and understand the edge cases.

Parsing. The tool reads the delimiters from each file's own ISA segment (element, component, terminator, line-break), so it handles `*` or `|` element separators, `~` or newline-only segment separators, and CRLF or LF. It splits the file into segments in their original order, keeping empty elements intact. It then groups the segments into the envelope/BGN header, subscribers (`INS*Y`), their dependents (`INS*N`), and coverage (`HD`/`DTP`/`REF`).

Matching subscribers. Each subscriber is given a key: - `REF*17` value, if present (the primary key); - otherwise the `NM1*IL` identity (last name / first name / member ID).

Subscribers are matched between the two files by this key: - in both -> compared in detail (Changed or Unchanged); - only in B -> Added; - only in A -> Removed.

Matching dependents. Dependents are matched the same way, by their `NM1*IL` identity. A dependent with no member ID falls back to name-only.

Matching segments (why reorders aren't false positives). Within a matched subscriber, segments are paired by a stable identity (segment type + the element that identifies it, e.g. the `REF` qualifier or the `DTP` code) rather than by their position in the file. That means if a segment moved to a different place, it still lines up with its counterpart, and only a genuine value change is reported.

Header differences. The envelope and BGN fields (ISA/GS/ST control numbers, dates, BGN reference, and other segments that appear before the first `INS`) are compared by field name and shown separately.

What it intentionally ignores. Loop 2700 / `LX` segments are not modelled — the validated real data contains none.

> The full list of assumptions and edge cases (coverage ordering, multi-`ST` overlap, NM1 ID location, anonymous subscribers) is documented in README.md under "Assumptions & edge cases".

---

9. Troubleshooting

Symptom	Likely cause / fix
`No module named 'tkinter'`	Install tkinter for your OS (see Section 1).
`No module named 'openpyxl'`	Run `pip install -r requirements.txt` in your active environment.
Error dialog: "no ISA segment at the start of the file"	The file doesn't start with an `ISA` segment. Confirm you picked a real 834 file (not a fragment or a differently-structured file).

Symptom	Likely cause / fix
Error dialog: "file is too short to contain a complete ISA segment"	The file is truncated or empty.
Error dialog: "nothing after the ISA segment"	The file has an ISA but no body.
A subscriber shows as Added and a matching one as Removed	The two files use different keys for the same person (e.g. one has `REF*17`, the other only `NM1*IL`). Check the Key column — if the keys differ, the tool can't know they're the same person.
A subscriber I expected to match is Unchanged when I thought it changed	Segment order differences are ignored by design. Look at the Detail column / detail pane to confirm only value changes are reported.
Export button does nothing / "Run a comparison first"	You must press Compare before Export to Excel….
Wrong files compared (Added/Removed look swapped)	You picked baseline and comparison the other way round. Swap File A and File B.

If something still looks wrong, the fastest diagnostic is the headless run (Section 7) — it prints the same counts the GUI shows, and you can share that output when reporting an issue.

---

10. Tips for large files

- The tool is built for scale. A file with 5,000 subscribers (~45,000 segments) parses in well under a second and compares in a few hundred milliseconds.
- Keep "Show all segments" off while scanning — it only renders the full side-by-side on demand for the subscriber you select, which keeps the UI snappy.
- Use the Excel report for review. The Subscriber Differences sheet is a flat, filterable list of every difference; the All Segments sheet is your complete audit trail.
- For batch work, use the headless mode (Section 7) in a loop over your file pairs.

---

11. Glossary

Term	Meaning
834	A HIPAA transaction set that carries benefit enrollment / coverage information.
ISA / GS / ST / SE / GE / IEA	The EDI "envelope" segments. ISA = inter-change (outermost), GS = functional group, ST/SE = a single transaction set, GE = end of functional group, IEA = end of inter-change.
BGN	The 834 "beginning of submission" segment — part of the file header.
Segment	One line of the EDI file, e.g. `NM1*IL*1*Smith*John****34*042640680~`.
Element	A value inside a segment, separated by the element delimiter (`*`).
`INS`	Identifies a member; `INS*Y` = the subscriber, `INS*N` = a dependent.
`REF*17`	A reference segment carrying the employee/reporting number — the primary subscriber key.
`NM1*IL`	A name segment for an individual — used for the subscriber/dependent identity fallback.
`HD` / `DTP`	Coverage (health plan) segments; `HD` marks the start of a subscriber's coverage block.
Baseline (File A)	The reference file; things missing in B are "Removed".
Comparison (File B)	The new file; things not in A are "Added".

---

12. Regenerating this guide (PDF)

This guide is authored in Markdown (`USER_GUIDE.md`) and rendered to `USER_GUIDE.pdf` by `make_pdf.py`. If you edit the Markdown, regenerate the PDF with:

```
pip install -r requirements-pdf.txt      # one-time (markdown + xhtml2pdf)
python make_pdf.py                       # USER_GUIDE.md -> USER_GUIDE.pdf
```

Notes: - The PDF tooling is optional — the app itself does not need it. - `make_pdf.py` embeds the fonts in `fonts/` (Segoe UI + Courier New) so arrows, dashes and the window diagram render correctly. If you move the project, make sure `fonts/segoeui.ttf` and `fonts/couri.ttf` exist (copy them from `C:\Windows\Fonts`). - The Markdown keeps the full Unicode (box-drawing, arrows, colour dots); the PDF build substitutes ASCII-safe equivalents for the diagram so it renders in any font.

See also: README.md (project overview & architecture), USER_GUIDE.pdf (this guide as a PDF), README_sanitizer.md (how the sample fixtures were made PII-free), JIRA_TICKET.md (build documentation).